

EXTEND WITH TOOLS

Scale to many tools with tool search

Scale your agent to thousands of tools by discovering and loading only what's needed, on demand.

Tool search enables your agent to work with hundreds or thousands of tools by dynamically discovering and loading them on demand. Instead of loading all tool definitions into the context window upfront, the agent searches your tool catalog and loads only the tools it needs.

This approach solves two challenges as tool libraries scale:

- **Context efficiency:** Tool definitions can consume large portions of the context window (50 tools can use 10-20K tokens), leaving less room for actual work.
- **Tool selection accuracy:** Tool selection accuracy degrades with more than 30-50 tools loaded at once.

Tool search is enabled by default.

How tool search works

When tool search is active, tool definitions are withheld from the context window. The agent receives a summary of available tools and searches for relevant ones when the task requires a capability not already loaded. The 3-5 most relevant tools are loaded into context, where they stay available for subsequent turns. If the conversation is long enough that the SDK compacts earlier messages to free space, previously discovered tools may be removed, and the agent searches again as needed.

Tool search adds one extra round-trip the first time Claude discovers a tool (the search step), but for large tool sets this is offset by smaller context on every turn. With fewer than ~10 tools, loading everything upfront is typically faster.

For details on the underlying API mechanism, see [Tool search in the API](#).

 Tool search is supported on every Claude model except Haiku.

Configure tool search

Tool search is on by default. It is disabled by default on Vertex AI, where it is supported for Claude Sonnet 4.5 and later and Claude Opus 4.5 and later. It is also disabled when `ANTHROPIC_BASE_URL` points to a non-first-party host, since most proxies do not forward `tool_reference` blocks. You can override either default with the `ENABLE_TOOL_SEARCH` environment variable:

Value	Behavior
(unset)	Tool search is on. Tool definitions are deferred and discovered on demand. Falls back to loading upfront on Vertex AI or a non-first-party <code>ANTHROPIC_BASE_URL</code> .
<code>true</code>	Tool search is always on. The SDK sends the beta header even on Vertex AI and through proxies. Requests fail on Vertex AI models earlier than Sonnet 4.5 or Opus 4.5, or on proxies that do not support <code>tool_reference</code> blocks.
<code>auto</code>	Checks the combined token count of all tool definitions against the model's context window. If they exceed 10%, tool search activates. If they're under 10%, all tools are loaded into context normally.
<code>auto:N</code>	Same as <code>auto</code> with a custom percentage. <code>auto:5</code> activates when tool definitions exceed 5% of the context window. Lower values activate sooner.
<code>false</code>	Tool search is off. All tool definitions are loaded into context on every turn.

Tool search applies to all registered tools, whether they come from remote MCP servers or **custom SDK MCP servers**. When using `auto`, the threshold is based on the combined size of all tool definitions across all servers.

Set the value in the `env` option on `query()`. This example connects to a remote MCP server that exposes many tools, pre-approves all of them with a wildcard, and uses `auto:5` so tool search activates when their definitions exceed 5% of the context window:

```

import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Find and run the appropriate database query",
  options: {
    mcpServers: {
      "enterprise-tools": {
        // Connect to a remote MCP server
        type: "http",
        url: "https://tools.example.com/mcp"
      }
    },
    allowedTools: ["mcp__enterprise-tools__*"], // Wildcard pre-approves all
tools from this server
    env: {
      ENABLE_TOOL_SEARCH: "auto:5" // Activate tool search when tools exceed
5% of context
    }
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}

```

Setting `ENABLE_TOOL_SEARCH` to `"false"` disables tool search and loads all tool definitions into context on every turn. This removes the search round-trip, which can be faster when the tool set is small (fewer than ~10 tools) and the definitions fit comfortably in the context window.

Optimize tool discovery

The search mechanism matches queries against tool names and descriptions. Names like `search_slack_messages` surface for a wider range of requests than `query_slack`. Descriptions with specific keywords (“Search Slack messages by keyword, channel, or date range”) match more queries than generic ones (“Query Slack”).

You can also add a system prompt section listing available tool categories. This gives the agent context about what kinds of tools are available to search for:

You can search for tools to interact with Slack, GitHub, and Jira.

Limits

- **Maximum tools:** 10,000 tools in your catalog
- **Search results:** Returns 3-5 most relevant tools per search
- **Model support:** every Claude model except Haiku

Related documentation

- **Tool search in the API:** Full API documentation for tool search, including custom implementations
- **Connect MCP servers:** Connect to external tools via MCP servers
- **Custom tools:** Build your own tools with SDK MCP servers
- **TypeScript SDK reference:** Full API reference
- **Python SDK reference:** Full API reference